



Distributed RAG-Based Web Scraper

Fault-tolerant distributed crawling · content versioning · pgvector semantic search · RAG question
answering with citations

UNIVERSITY FINAL PROJECT

OVERVIEW

What it does

- Crawls multiple sites — static, JS-rendered, and a 500+ page catalog — using many independent worker containers coordinated through a shared job queue.
- Cleans HTML to Markdown and versions every change — nothing is ever overwritten.
- Chunks, embeds, and indexes content into pgvector for semantic search.
- Serves keyword, semantic, and hybrid search over the crawled corpus.
- Answers questions with a RAG pipeline that streams an answer and cites every claim.
- Demonstrates horizontal scaling and graceful recovery from worker failure.

ARCHITECTURE

Two decoupled halves, one database

- INGESTION — the slow, distributed, fault-tolerant path: crawl → clean → version → chunk → embed → index.
- SERVING — the fast, read-mostly path: keyword / semantic / hybrid search, and RAG question answering.
- The two never talk directly; they meet in Postgres. Crawling and searching run concurrently — freshly crawled pages appear in results as they finish indexing.
- Coordination (queues + per-crawl counters) lives entirely in Redis, so workers share nothing and scale horizontally.

ARCHITECTURE

The services

SERVICE	ROLE
api (Fastify)	HTTP: search, streaming ask, admin; enqueues crawls
worker × N (BullMQ)	Runs scrape / discover / index jobs — horizontally scalable
web (Next.js 15)	Public search + ask UI, and the operator admin panel
scheduler	Sweeps every 5 min to rescue stuck crawl runs
Postgres + pgvector	Sources, pages, versions, chunks, and embedding vectors
Redis	Job queues + per-crawl coordination counters

TECH STACK

Locked choices & why

CONCERN	CHOICE	WHY
Queue	BullMQ on Redis	Built-in retries, backoff, rate limiting, DLQ
Scraper	Crawlee (Cheerio + Playwright)	One API for static + JS pages
DB / vectors	Postgres 16 + pgvector	One store, HNSW index, no extra service
Embeddings	OpenAI text-embedding-3-small	Cheap, strong, 1536-dim
LLM	OpenAI GPT-5.5	Good citation adherence; one provider
API / UI	Fastify · Next.js 15 · Tailwind	TS-native, SSR + streaming

KEY DESIGN DECISION

BullMQ owns coordination. Crawlee is just the fetcher.

Crawlee's built-in RequestQueue is in-process — it cannot coordinate across containers. Using it as the coordinator would make horizontal scaling impossible. Instead, every discovered link is handed back to BullMQ, not Crawlee. That single decision is what makes the “distributed” story real.

INGESTION

The crawl lifecycle

- Start crawl → API opens a tracked CrawlRun, reserves the seed URL, enqueues one scrape job.
- scrape — cancel check → robots.txt → per-domain rate limit (non-blocking defer) → fetch (Cheerio or Playwright).
- Clean to Markdown, hash it, compare to the latest version: unchanged → skip; changed/new → write a new PageVersion.
- Fan out: filtered links → a discover job; the saved version → an index job.
- discover reserves each genuinely-new link and enqueues its scrape job — the loop spreads across all workers.

INGESTION

Processing & versioning

- Readability strips boilerplate; Turndown converts to Markdown; tables are extracted separately as structured JSON.
- Content dedup by SHA-256 of the cleaned Markdown — an unchanged re-crawl costs nothing (no re-embed).
- Every change is a NEW version; history is never destroyed — this powers the admin diff viewer.
- URL dedup via a per-run “seen” set, so no page is crawled twice in one run.

INGESTION

Indexing: text → searchable vectors

- Heading-aware chunking (~800 tokens, 150 overlap); tables become their own chunks with a caption.
- Chunks are embedded in batches via OpenAI text-embedding-3-small (1536-dim).
- Stored in Postgres via raw SQL with a pgvector column (HNSW, cosine) + a generated tsvector (GIN) for keyword search.
- Re-indexing a changed page deletes the old version's chunks — only the latest version is searchable; old ones remain for history.

SERVING

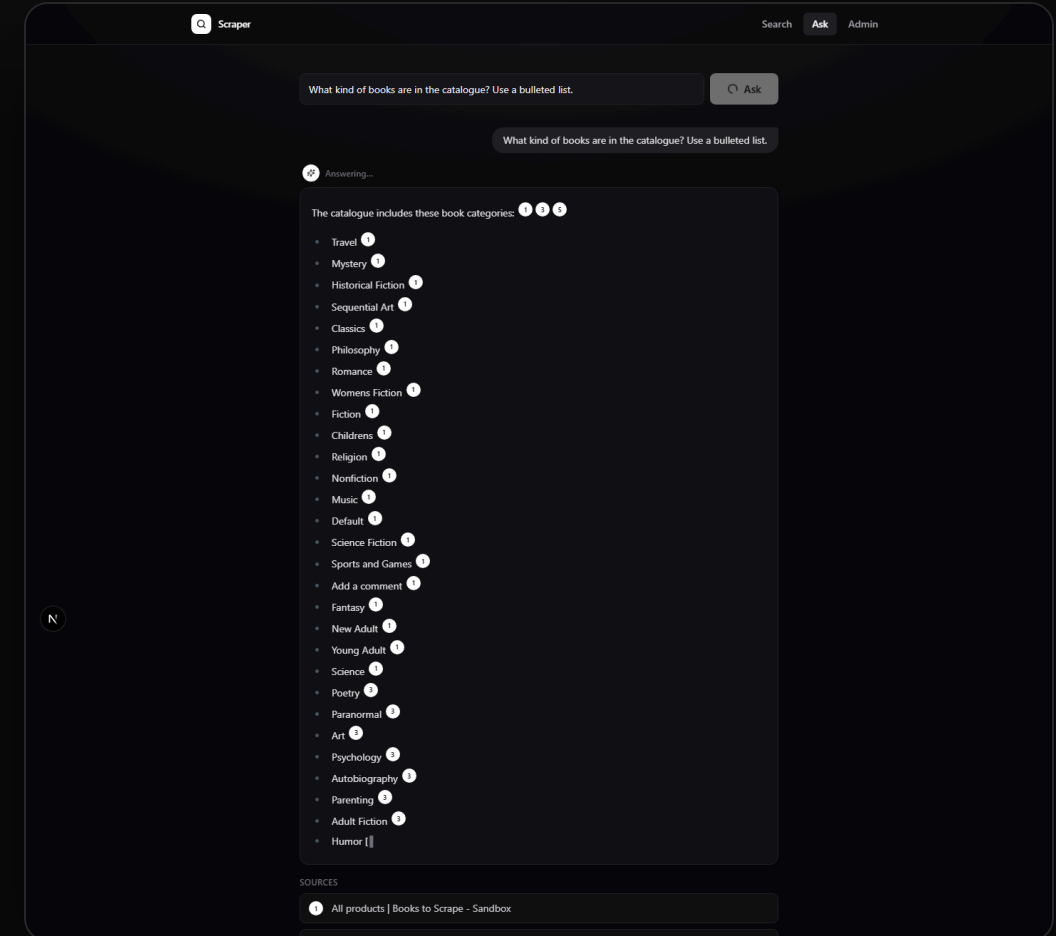
Retrieval: three modes

- keyword — Postgres full-text search over the tsvector column. Fast, exact-term.
- semantic — pgvector cosine similarity over the embedded query. Finds meaning, not words.
- hybrid (default) — Reciprocal Rank Fusion (k=60) of both lists, since a keyword score and a cosine distance aren't on comparable scales.
- All three share one code path and an optional per-source filter.

SERVING

RAG with citations

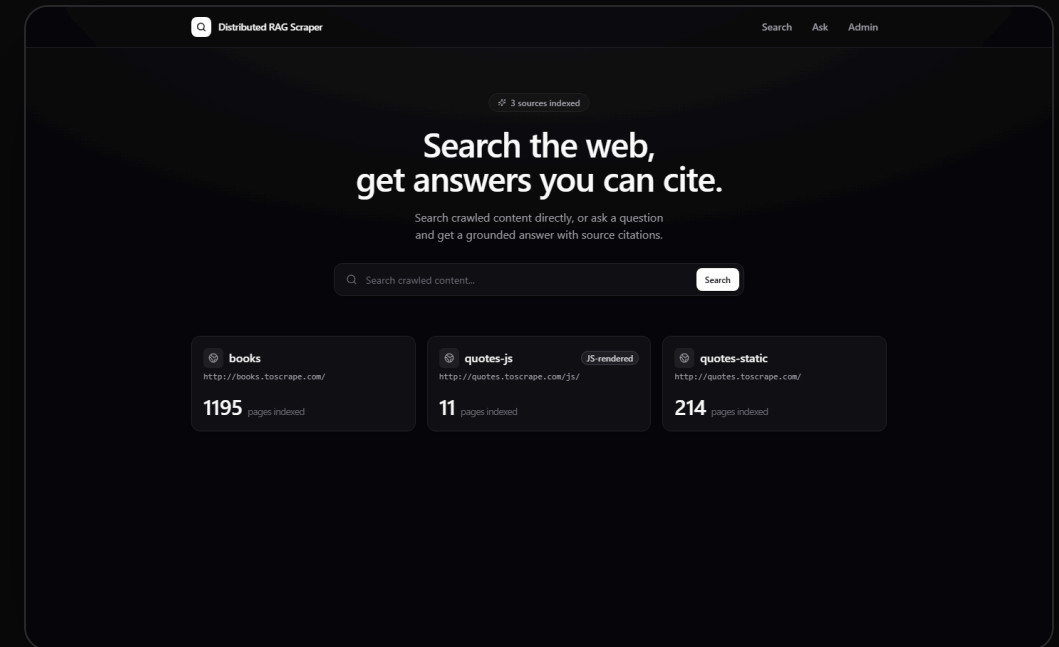
- Retrieve top chunks → build ONE citation-forcing prompt → stream GPT-5.5 over SSE.
- Every claim must cite [n]; the answer streams token-by-token with a live cursor.
- Each [n] is a clickable chip that jumps to the exact cited chunk on its source page.
- Guarantee: a citation can never point at a URL the model invented — the list is built from the actual retrieved chunks.



PRODUCT

The interface

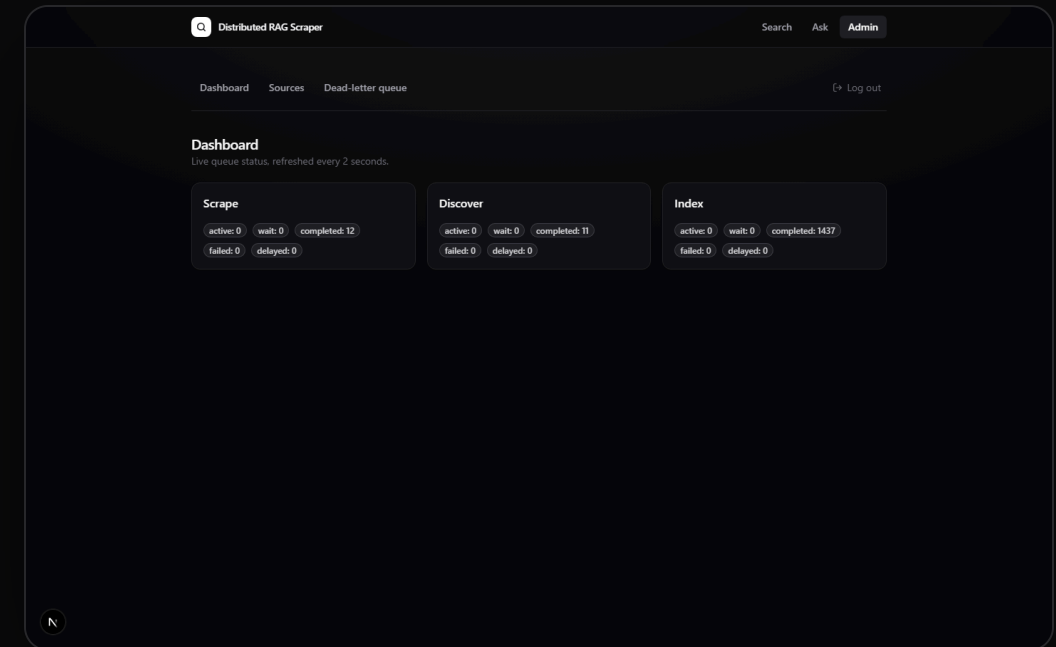
- Public: a landing page with live source stats, a search page with mode + source filters, and the streaming ask page.
- Admin (token-gated): live queue dashboard, source management + start-crawl, dead-letter-queue retry, and a word-level version diff viewer.
- Next.js 15 App Router, SSR + streaming, a dark hand-built design system.



OPERATIONS

Operator dashboard

- Live queue counters (scrape / discover / index) poll every 2 seconds.
- Dead-letter queue: jobs that exhausted retries are visible and one-click retryable — nothing fails silently.
- Per-source crawl progress and history, driven by the tracked CrawlRun counters.



RELIABILITY

Fault tolerance

- Retries: every job retries 5× with exponential backoff (2s → 32s); terminal failures land in the DLQ.
- Worker death: a killed worker's in-flight jobs are detected as stalled (~30s) and re-run by a survivor.
- Fleet-wide politeness: the per-domain rate limit and origin-pushback cooldowns live in Redis — adding workers never hammers a site harder.
- Stuck-run safety net: the scheduler force-finalizes any run silent for 30+ minutes.

RESULTS

Horizontal scaling (measured)

WORKERS	CRAWL TIME	PAGES/SEC	SPEEDUP
1	52.8 s	4.05	1.00×
2	30.6 s	6.99	1.73×
4	19.4 s	11.03	2.72×

214-page crawl, rate cap raised so the queue — not politeness — is the bottleneck. Sublinear because every worker updates one hot CrawlRun counter row.

CHAOS ENGINEERING

The chaos test found a real bug — and proved the fix.

Before: SIGKILL 1 of 4 workers mid-crawl → run stuck at 213/214, one URL orphaned between reserve and enqueue. Fix: split “seen” (counted) from “enqueued” (job confirmed) so a retry re-creates the lost job; reconcile the queued counter from Redis. After (rebuilt image): 214/214, settled == queued, zero lost jobs — verified end-to-end.

HONESTY

Known limitations

- Compound questions: single-shot top-K retrieval can starve one half of a two-part question — the stronger match crowds out the other.
- Citations prove a source was retrieved, not that the model characterized it correctly — it can misdescribe a real cited source.
- RAG answers are single-turn (no conversation memory); LLM latency is variable.
- Documented, reproduced, and each with a considered fix — see [docs/limitations.md](#).

Distributed, fault-tolerant, and honest about its limits.

- Distribution that's real: shared-nothing workers, Redis-coordinated, chaos-verified.
- Content integrity: full version history, content-hash dedup, no silent overwrites.
- Grounded answers: hybrid retrieval + citation-forced RAG that can't fabricate sources.

THANK YOU